



**CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
CST ANÁLISE E DESENVOLVIMENTO DE SISTEMAS - DISTÂNCIA (2701)
DESENVOLVIMENTO WEB BACK END**

**ATIVIDADE PRÁTICA
API REST - BAOZI STORE**

APOLLO HOINATZ BARDINI - RU: 5586086

NOVA ESPERANÇA DO SUDOESTE - PR

2026

SUMÁRIO

1. Descrição de uma situação fictícia
2. Diagrama de Caso de Uso
3. Especificação da API desenvolvida
4. Prints do Postman
5. Link do repositório (GitHub ou similar)

1. Descrição de uma situação fictícia

A Baozi Store é uma pequena loja especializada na venda de pão chinês. Para facilitar o controle do negócio, foi desenvolvida uma API REST para organizar os cadastros de clientes, produtos e pedidos.

O cliente APOLLO HOINATZ BARDINI5586086 realizou seu cadastro no sistema em 17/08/2026. O produto utilizado no estudo de caso é o Baozi Tradicional, vendido por unidade pelo valor de R\$ 12,50 e disponível em estoque. Em seguida, o cliente realizou um pedido de 3 unidades desse produto.

A aplicação registra o cliente, o produto comprado e a quantidade solicitada, permitindo consultar e excluir os registros por meio dos endpoints implementados.

Identificação utilizada nos testes: APOLLO HOINATZ BARDINI5586086

2. Diagrama de Caso de Uso

O diagrama apresenta o ator principal, Usuário da API, e as operações realizadas nas entidades Cliente, Produto e Pedido. Além das operações obrigatórias, a atualização por PUT foi implementada como recurso adicional.

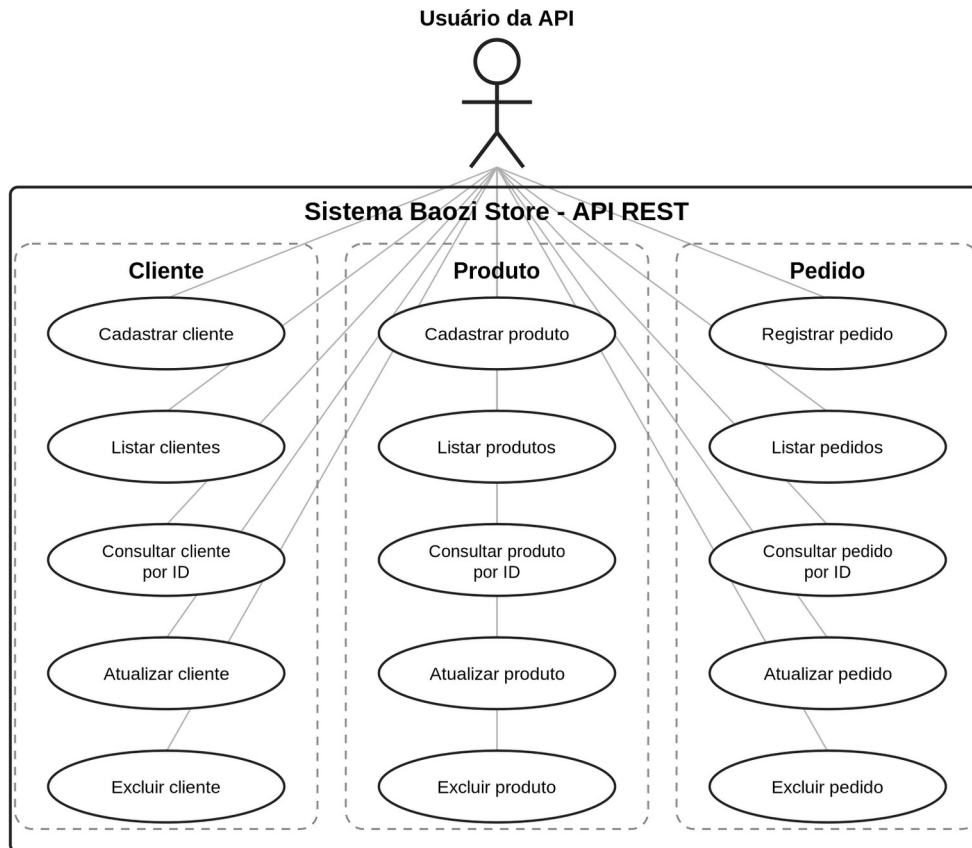


Figura 1 - Diagrama de Caso de Uso da API REST Baozi Store.

3. Especificação da API desenvolvida

A aplicação foi desenvolvida em Java 17 com Spring Boot. A persistência utiliza Spring Data JPA e banco relacional H2. A organização do projeto segue a estrutura mínima solicitada, com os pacotes model, repository e controller.

3.1 Entidades e campos

Cliente

Campo	Tipo	Descrição
id	Long	Identificador do cliente
nome	String	Nome do cliente
clienteDesde	LocalDate	Data de início do cliente

Produto

Campo	Tipo	Descrição
id	Long	Identificador do produto
nome	String	Nome do produto
preco	BigDecimal	Preço unitário
estoque	Boolean	Indica disponibilidade em estoque

Pedido

Campo	Tipo	Descrição
id	Long	Identificador do pedido
clienteId	Long	ID do cliente
produtoId	Long	ID do produto
quantidade	Integer	Quantidade comprada

3.2 Endpoints implementados

Método	Endpoint	Finalidade
POST	/clientes	Criar cliente
GET	/clientes	Listar clientes
GET	/clientes/{id}	Consultar cliente por ID
PUT	/clientes/{id}	Atualizar cliente (adicional)
DELETE	/clientes/{id}	Excluir cliente
POST	/produtos	Criar produto
GET	/produtos	Listar produtos
GET	/produtos/{id}	Consultar produto por ID
PUT	/produtos/{id}	Atualizar produto (adicional)
DELETE	/produtos/{id}	Excluir produto
POST	/pedidos	Criar pedido
GET	/pedidos	Listar pedidos
GET	/pedidos/{id}	Consultar pedido por ID
PUT	/pedidos/{id}	Atualizar pedido (adicional)
DELETE	/pedidos/{id}	Excluir pedido

3.3 Exemplos de JSON

Cliente - POST /clientes

```
{  
  "nome": "APOLLO HOINATZ BARDINI5586086",  
  "clienteDesde": "2026-08-17"  
}
```

Produto - POST /produtos

```
{  
  "nome": "Baozi Tradicional",  
  "preco": 12.50,  
  "estoque": true  
}
```

Pedido - POST /pedidos

```
{  
  "clienteId": 1,  
  "produtoId": 1,  
  "quantidade": 3  
}
```

3.4 Organização do projeto

- model: entidades JPA Cliente, Produto e Pedido.
- repository: interfaces JpaRepository para acesso aos dados.
- controller: endpoints REST das três entidades.
- resources/application.properties: configuração da aplicação, JPA e banco H2.

4. Prints do Postman

Os testes abaixo foram realizados com a aplicação em execução local na porta 8080. Para cada entidade são apresentados os testes de criação, listagem geral, consulta por ID e exclusão.

Entidade	Operações registradas	Status HTTP observados
Cliente	POST, GET geral, GET por ID, DELETE	201, 200, 200, 204
Produto	POST, GET geral, GET por ID, DELETE	201, 200, 200, 204
Pedido	POST, GET geral, GET por ID, DELETE	201, 200, 200, 204

4.1 Cliente - criação e listagem

POST - Criar Cliente

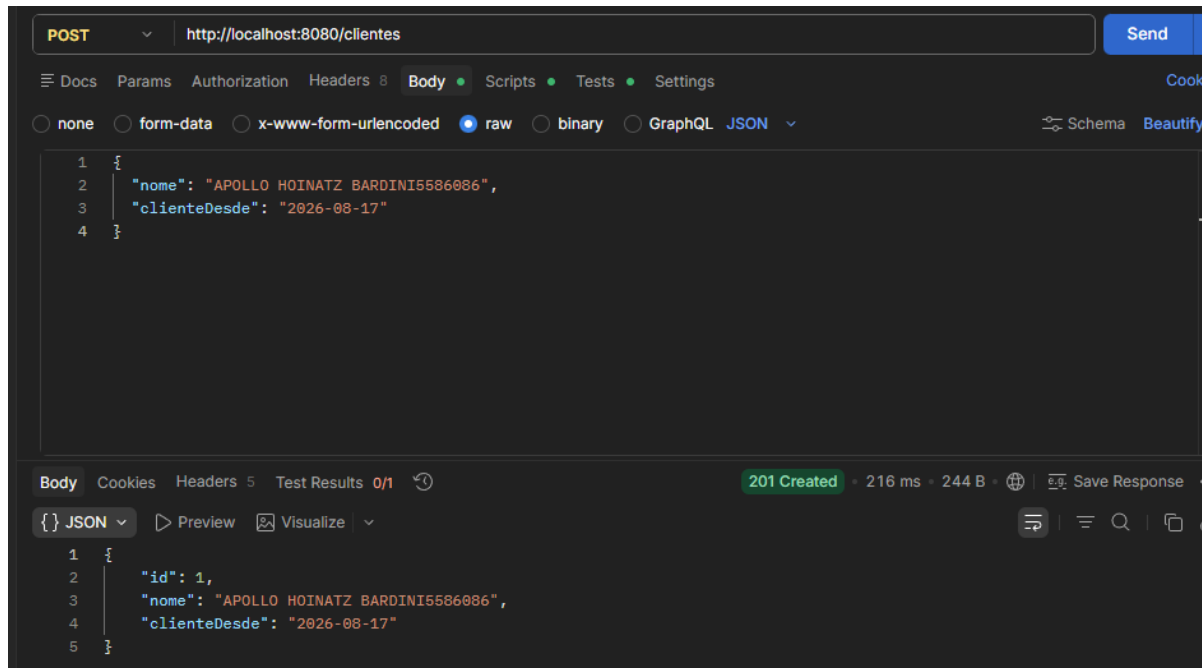


Figura 2 - POST - Criar Cliente.

GET - Listar Clientes

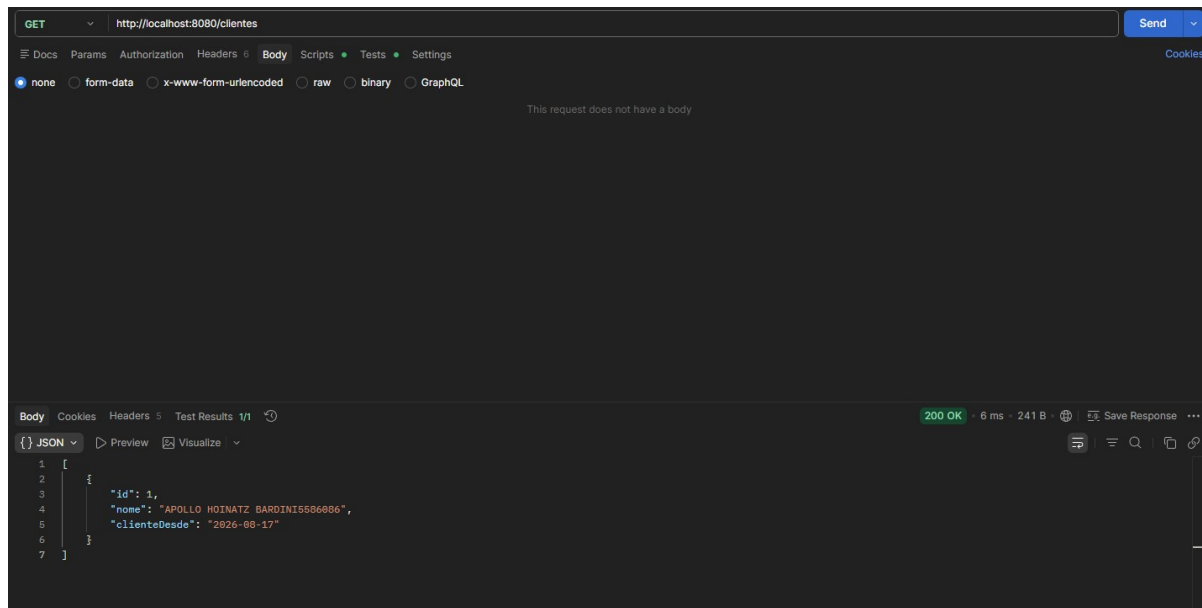


Figura 3 - GET - Listar Clientes.

4.2 Cliente - consulta por ID e exclusão

GET - Cliente por ID

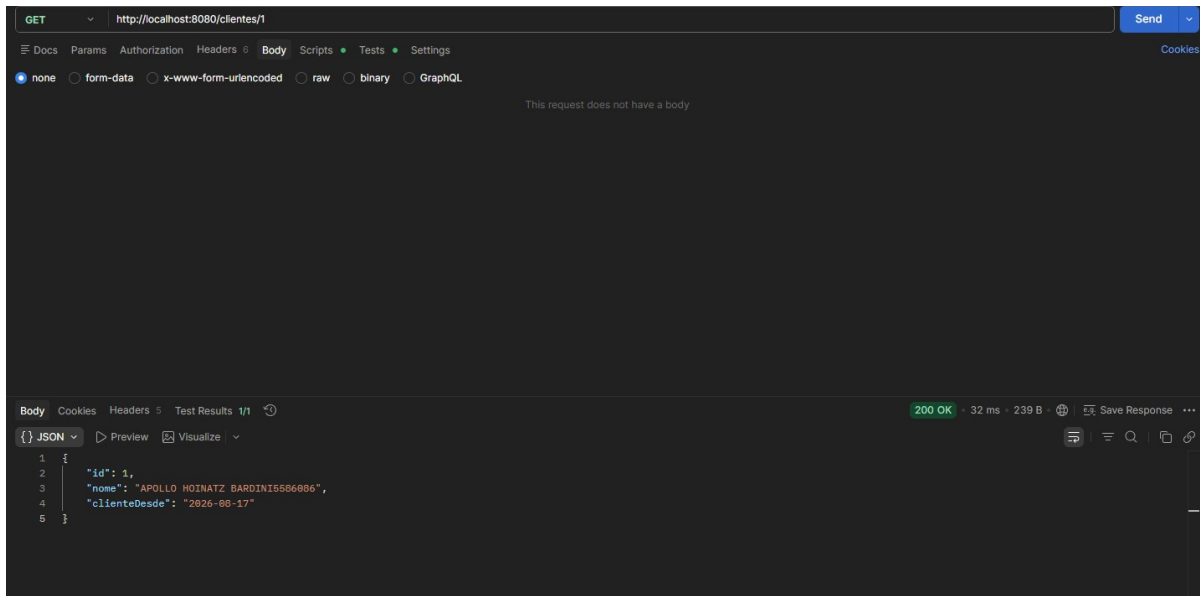


Figura 4 - GET - Cliente por ID.

DELETE - Cliente

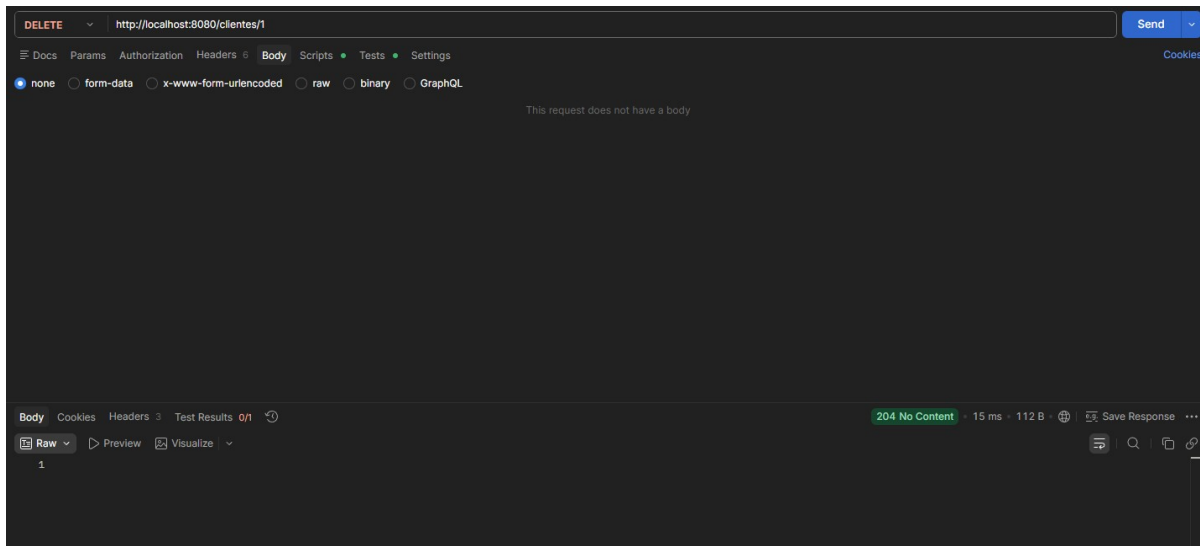


Figura 5 - DELETE - Cliente.

4.3 Produto - criação e listagem

POST - Criar Produto

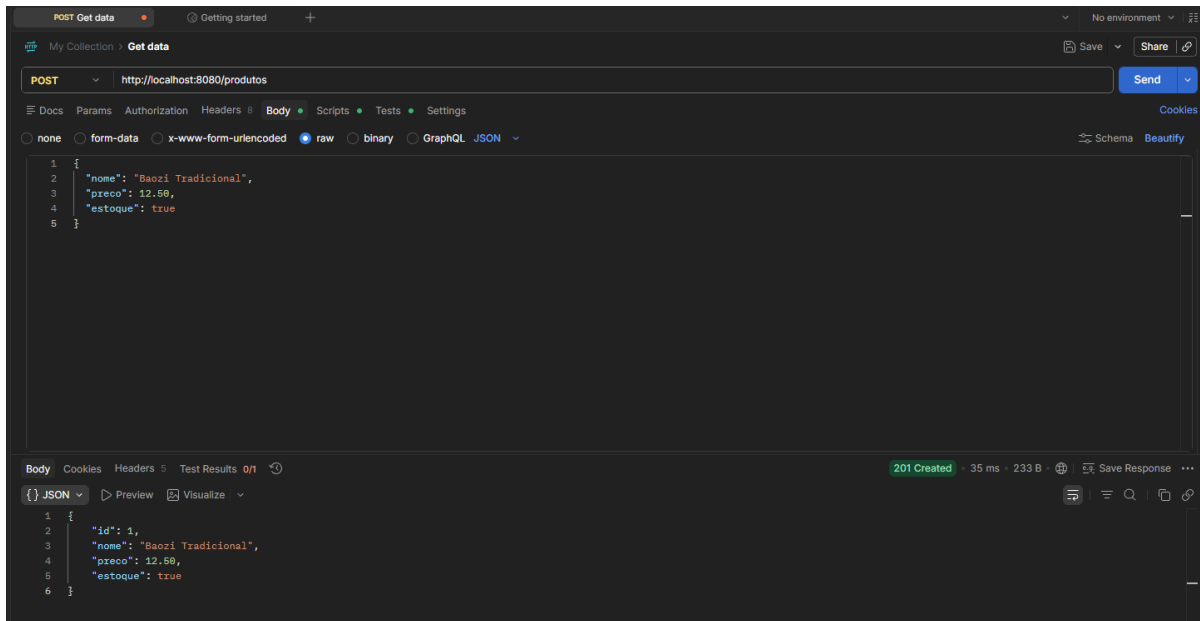


Figura 6 - POST - Criar Produto.

GET - Listar Produtos

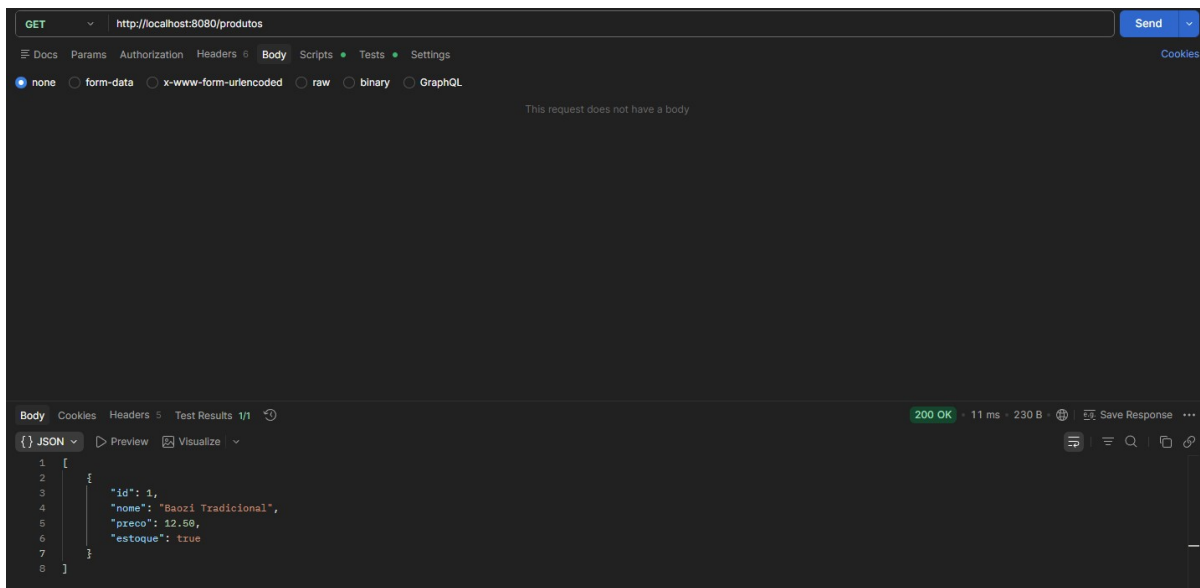


Figura 7 - GET - Listar Produtos.

4.4 Produto - consulta por ID e exclusão

GET - Produto por ID

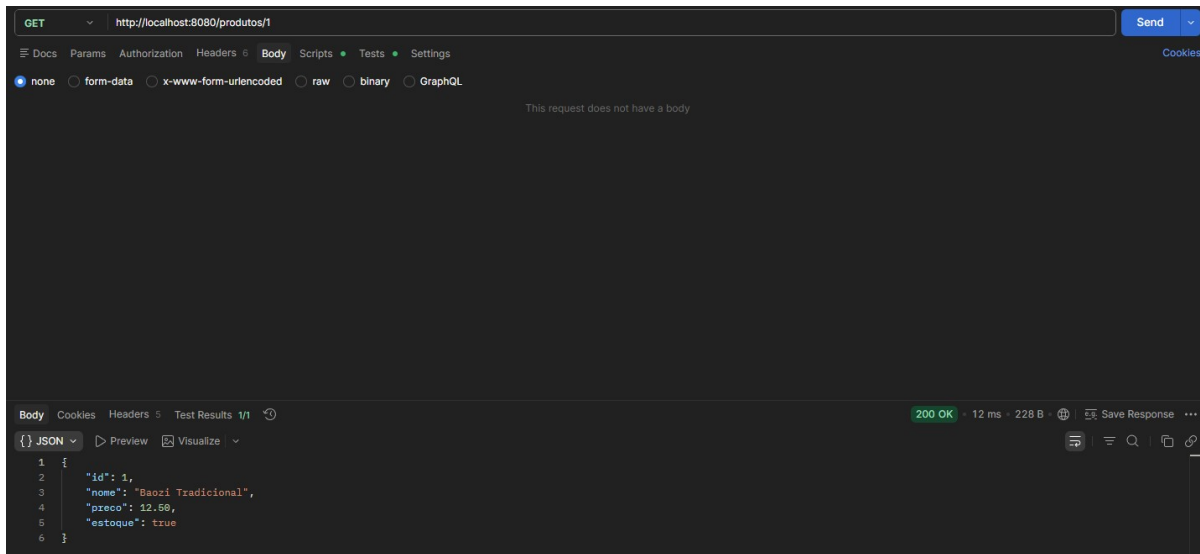


Figura 8 - GET - Produto por ID.

DELETE - Produto

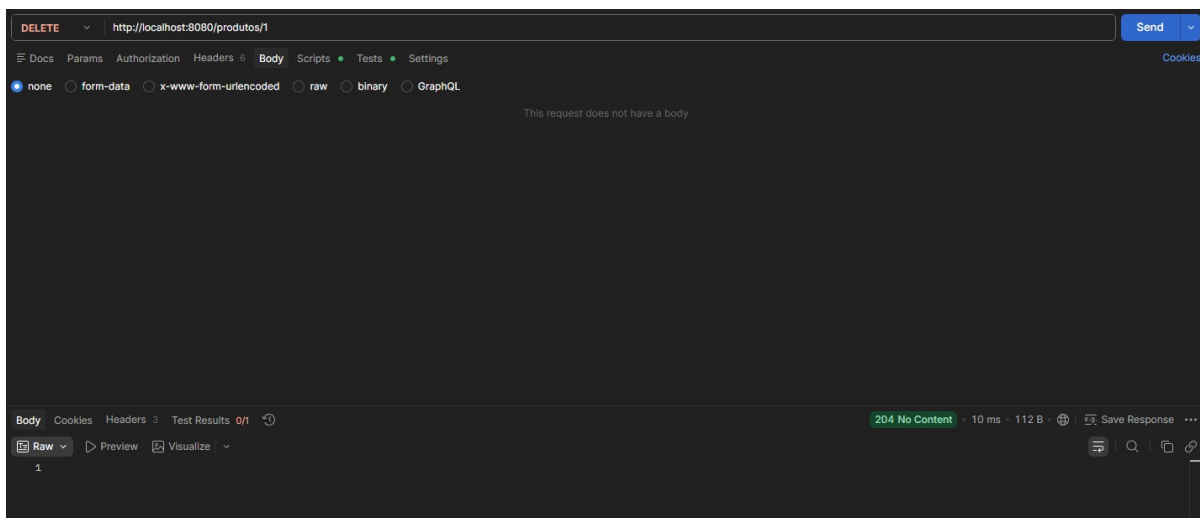


Figura 9 - DELETE - Produto.

4.5 Pedido - criação e listagem

POST - Criar Pedido

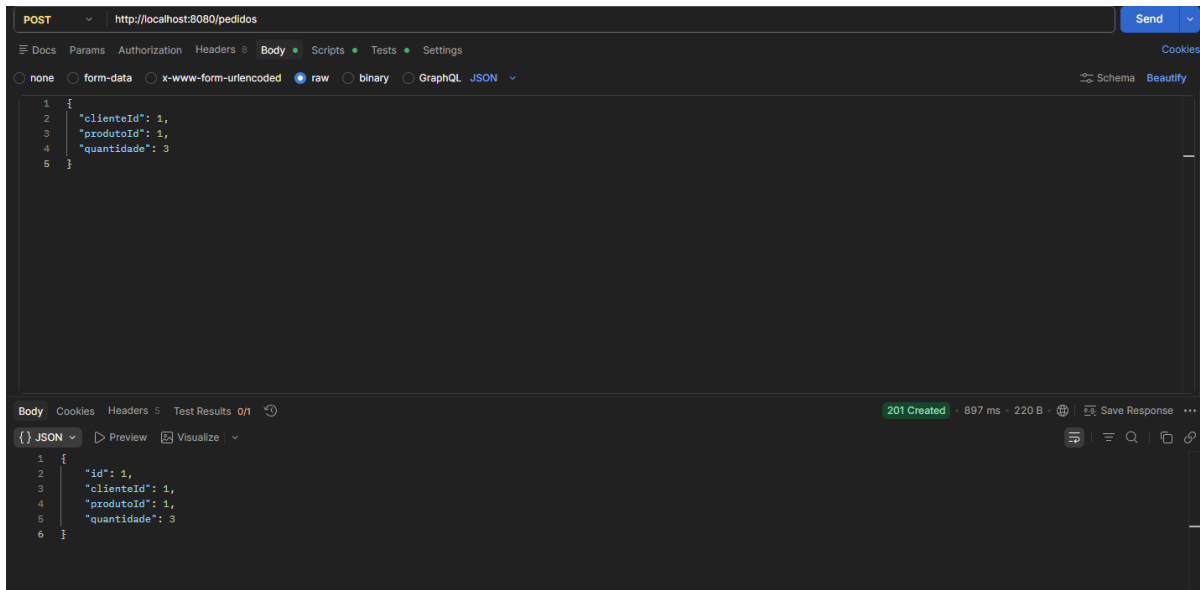


Figura 10 - POST - Criar Pedido.

GET - Listar Pedidos

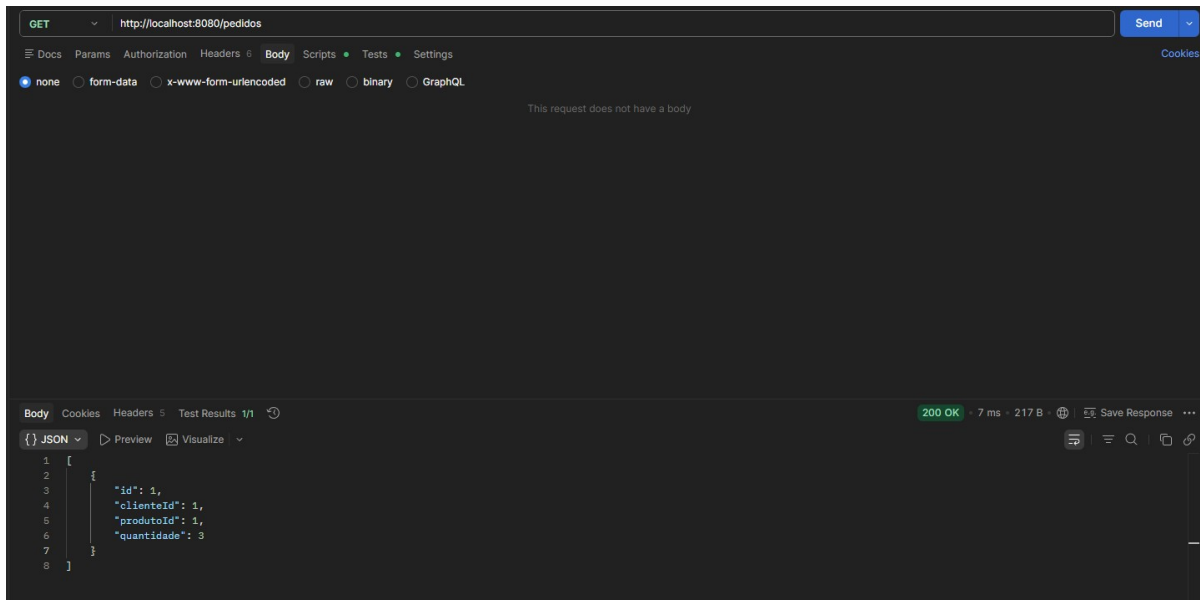


Figura 11 - GET - Listar Pedidos.

4.6 Pedido - consulta por ID e exclusão

GET - Pedido por ID

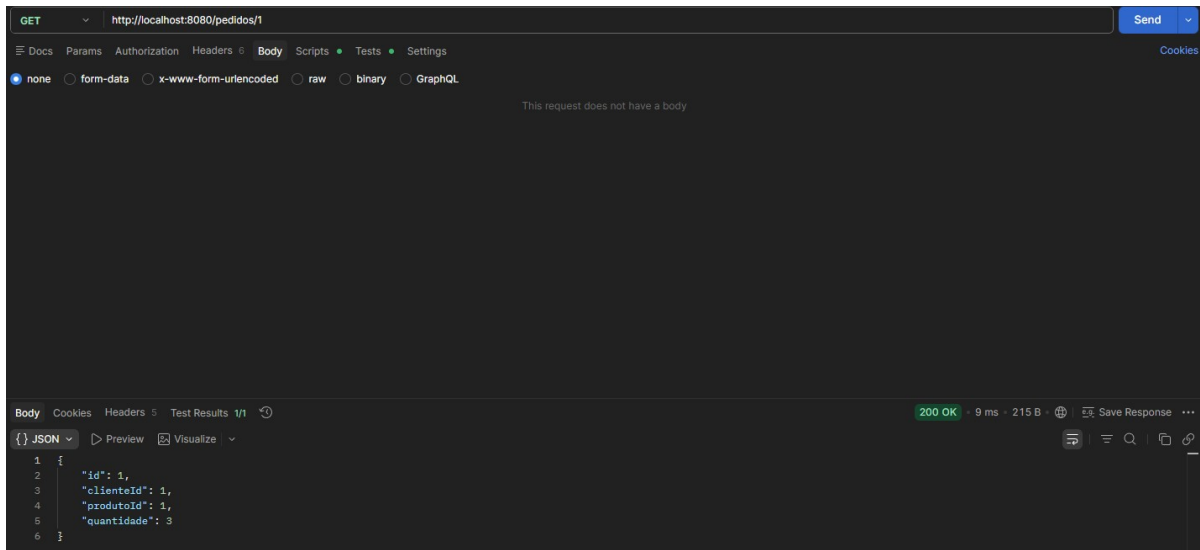


Figura 12 - GET - Pedido por ID.

DELETE - Pedido

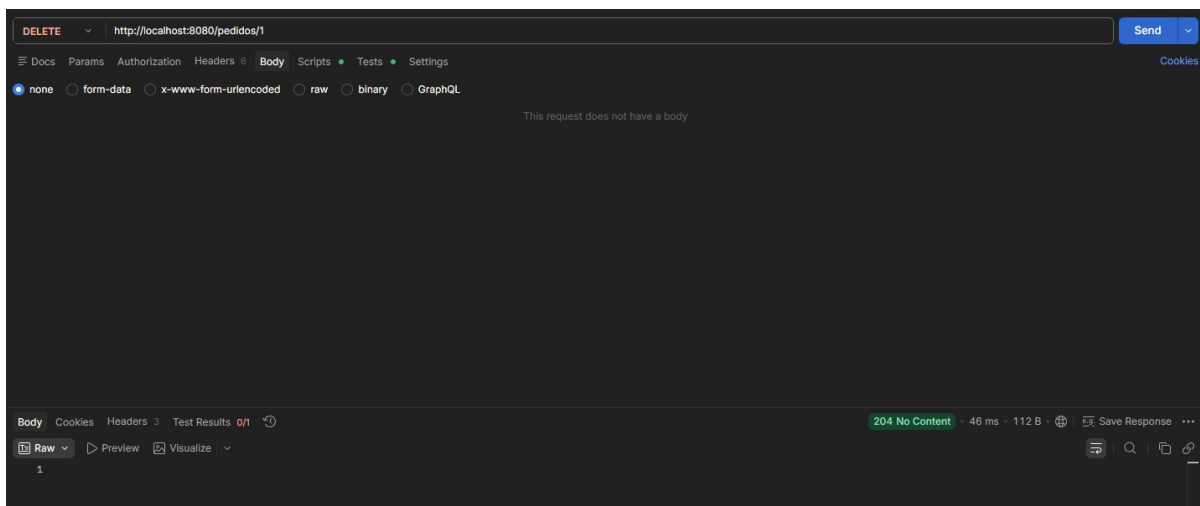


Figura 13 - DELETE - Pedido.

5. Link do repositório (GitHub ou similar)

O código-fonte do projeto está disponível no repositório público abaixo:

Link do repositório: <https://github.com/Apollo-Hb/baozi-store-api>

O repositório contém o código-fonte da API REST, o arquivo pom.xml, a configuração do banco H2 e a coleção do Postman utilizada nos testes.